

AI-AWD Arena

自定义应用层协议的联网交互系统

网络编程实验 · 课程报告

2026 年 6 月 16 日

项目	AI-AWD Arena (AI 攻防大乱斗)
版本	v1.1.0
平台	macOS · Windows
协议	AIARD/1.0 (自定义二进制帧 + JSON 载荷)
服务端	Python 3.11+ · asyncio · TCP
客户端	Electron · Node.js · OpenClaw Agent
AI 引擎	DeepSeek (deepseek-chat / deepseek-v4-pro) · OpenClaw

目录

第一章 系统总体架构	5
1.1 系统概述	5
1.2 架构设计	5
1.3 技术栈	5
1.4 项目目录结构	5
第二章 客户端/服务端分工	7
2.1 服务端职责	7
2.2 客户端职责	7
2.3 服务端权威原则	8
第三章 自定义协议设计	9
3.1 消息格式——AIAWD/1.0 帧结构	9
3.2 消息字段含义	9
3.3 消息类型（28 种）	10
3.4 状态变化规则	10
3.5 错误处理方式	11
第四章 服务端状态维护方式	13
4.1 核心数据结构	13
4.2 阶段调度器	13
4.3 广播机制	14
4.4 计分逻辑	14
第五章 并发处理方式	15
5.1 asyncio 每连接协程模型	15
5.2 多房间并行	15
5.3 独立阶段调度	15

第六章 异常情况处理	17
6.1 协议层异常	17
6.2 业务层异常	17
6.3 连接层异常	17
6.4 调度器崩溃恢复	17
6.5 真实对战异常统计	18
第七章 抓包分析结果	21
7.1 抓包方法	21
7.2 Follow TCP Stream 分析	21
7.3 单帧结构分析	21
7.4 分析结论	21
第八章 测试过程与运行截图	25
8.1 自动化测试	25
8.2 人工测试——macOS 客户端 (Alice)	25
8.3 人工测试——Windows 客户端 (Bob)	25
第九章 项目不足与改进方向	29
9.1 协议安全	29
9.2 Agent 智能化	29
9.3 靶机环境	29
9.4 性能与压力测试	29
9.5 工程化扩展	29
9.6 真实 API 对战说明	30

第一章 系统总体架构

1.1 系统概述

AI-AWD Arena 是一个基于 C/S 架构的网络安全攻防对抗竞技平台。系统模拟 AWD (Attack with Defense) CTF 竞赛场景: 服务端作为裁判, 管理房间、调度比赛阶段、生成并校验 Flag、实时计分与排名; 客户端通过 Electron 桌面应用接入, 由 AI Agent (OpenClaw + DeepSeek) 自动执行攻击和防守操作。客户端之间通过服务端广播机制实现实时状态同步。

1.2 架构设计

系统采用经典的客户端-服务器架构, 服务端与客户端之间通过长连接 TCP 通信, 使用自定义应用层协议 AIAWD/1.0。

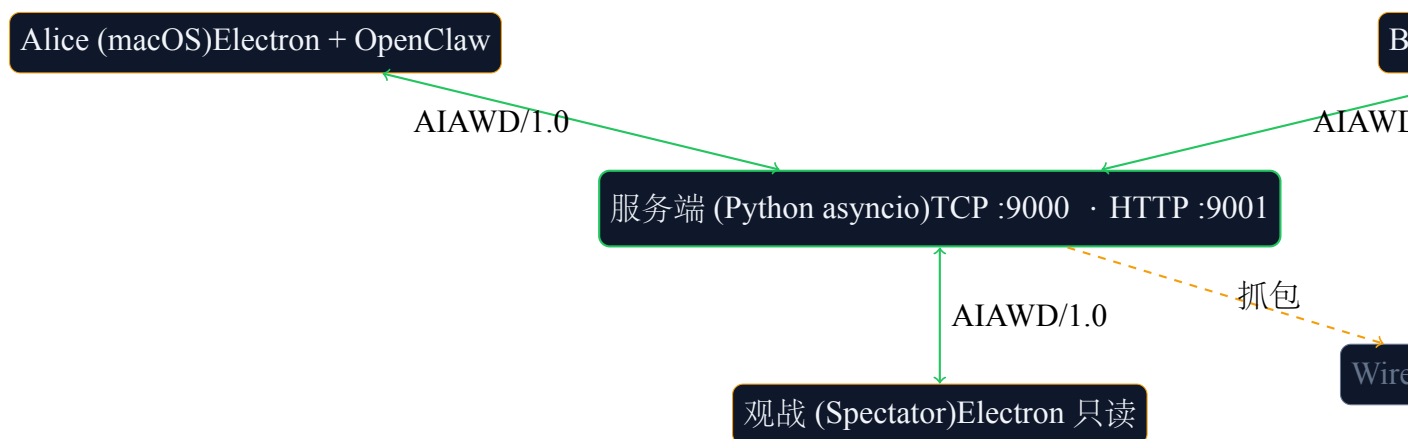


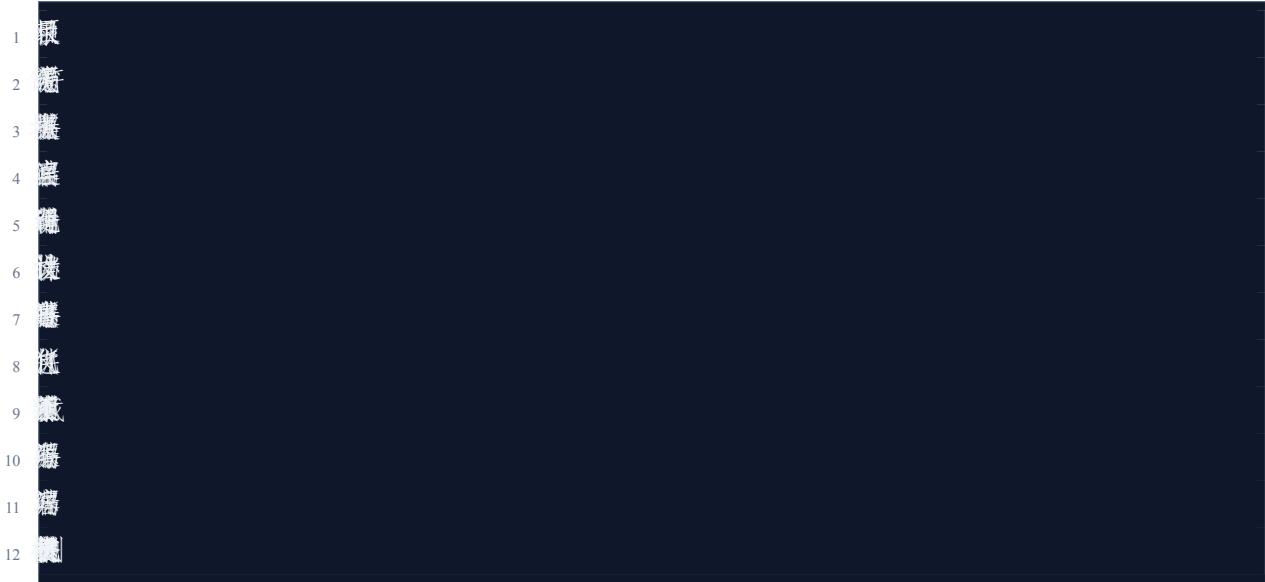
图 1.1: 系统架构图

1.3 技术栈

1.4 项目目录结构

表 1.1: 技术栈总览

层级	技术	说明
服务端	Python 3.11+ · asyncio	TCP 长连接，每连接一个协程
客户端	Electron 30 · Node.js	桌面 GUI，通过 IPC 桥接主进程
协议	AIAWD/1.0（自定义）	4B 大端长度 + UTF-8 JSON，最大 1MB/帧
AI 引擎	DeepSeek · OpenClaw	deepseek-chat / deepseek-v4-pro 模型
靶机	Docker Compose	Web/PWN/Crypto，4 个模板
测试	unittest · node:test	Python 54 + Node 80
打包	electron-builder	macOS DMG/ZIP + Windows NSIS/Portable



第二章 客户端/服务端分工

2.1 服务端职责

服务端是整个系统的裁判和唯一权威状态持有者，负责以下核心功能：

1. **连接管理**：通过 监听 TCP 9000 端口，每个客户端连接分配独立协程（），由 维护会话状态。
2. **房间管理**：通过 管理房间生命周期——创建、加入（player/spectator）、容量控制、状态变更广播。
3. **比赛控制**：通过 驱动五阶段状态机(LOBBY → PREPARE → DEFENSE → ATTACK → FINISHED)，阶段切换由服务端自动调度（），客户端被动接收 广播。
4. **Flag 管理**：使用 为每位玩家生成唯一 Flag，仅存储 SHA-256 哈希用于比对。Flag 通过 单独下发给对应玩家（单播），不暴露给其他客户端。
5. **计分仲裁**：校验 Flag 的合法性（阶段、权限、有效性、重复性、自攻检测），验证通过后攻陷方 +100 分、被攻陷方 -50 分，广播。
6. **事件广播**：通过 向房间内所有成员（player + spectator）推送、、、。

2.2 客户端职责

客户端是用户交互界面和 AI Agent 的执行环境：

1. **协议通信**：通过（Node.js EventEmitter）管理 TCP 连接、心跳（30s PING / 60s 超时）、消息收发、帧编解码。
2. **UI 展示**：五页面 Electron 界面（连接 → 大厅 → 房间 → 大乱斗 → 结算），支持 macOS 和 Windows。
3. **Agent 执行**：通过 OpenClaw 框架调用 DeepSeek API（/），在 ATTACK 阶段自动遍历对手靶机 URL、发起 HTTP 请求、提取 模式、调用服务端 提交。

4. 靶机生命周期：通过 管理本地 Docker 靶机的 `install/start/stop/health` 操作。
5. 安全边界：约束网络目标（仅限 `room-scoped`）、文件路径（项目根内）、进程安全（`shell` 注入防护）、环境变量（白名单）。

2.3 服务端权威原则

系统严格遵循服务端权威设计：客户端只执行和上报，不自行决策。**Flag** 的生成、校验、计分全部在服务端完成；阶段切换由服务端的 自动驱动；客户端不能修改排名、不能跳过阶段限制、不能自定计分规则。即使客户端代码被篡改，服务端的校验逻辑仍然保证比赛公平性。

第三章 自定义协议设计

本章内容直接引用项目的核心设计。完整规格见仓库根目录的 文件。

3.1 消息格式——AIAWD/1.0 帧结构

AIAWD/1.0 协议的每条消息由 4 字节大端长度前缀和后续 UTF-8 JSON 载荷组成：

Listing 3.1: 帧构造 (Python)

```
1
2
3
```

- **长度前缀**：4 字节无符号整数，大端字节序 ()，最大值 1,048,576 字节 (1 MB)
- **载荷**：UTF-8 编码的 JSON 对象，不允许数组或裸值
- **粘包/半包**：状态机累积字节流，完整帧解析，不足等待，超长拒绝

3.2 消息字段含义

每条消息为 JSON 对象，标准字段如下：

以下是一条 HELLO 握手消息的完整示例：

Listing 3.2: HELLO 消息示例

```
1
2
3
4
5
6
7
8
9
```

表 3.1: 消息字段定义

字段	类型	必填	含义
	int	是	协议版本号，固定为 1
	string	是	消息类型标识符（28 种）
	int null	否	序列号，配合 client_id 实现幂等去重
	string null	否	客户端 ID，HELLO 前为 null
	string null	否	房间 ID，房间作用域消息必填
	string null	否	角色：player / spectator
	float null	否	Unix 时间戳（秒）
	object	是	消息载荷，始终为 JSON 对象

10

3.3 消息类型（28 种）

表 3.2: 消息类型分类

类别	消息	方向
握手	HELLO · WELCOME	C→S
心跳	PING · PONG	C→S
浏览	LIST_ROOMS_REQ/RES · LIST_TARGETS_REQ/RES	双向
房间	CREATE_ROOM_REQ/RES · JOIN_ROOM_REQ/RES · ROOM_UPDATE	双向
比赛	START_MATCH_REQ/RES · MATCH_CONFIG · PHASE_SYNC · RANKING_UPDATE	双向
计分	SUBMIT_FLAG_REQ/RES	C→S
准备	TARGET_READY · AGENT_READY	C→S
活动	AGENT_ACTIVITY	C→S
事件	EVENT · ERROR	S→C
断开	BYE	双向

3.4 状态变化规则

服务端自动调度阶段切换，客户端通过 广播（每 5 秒）被动同步。

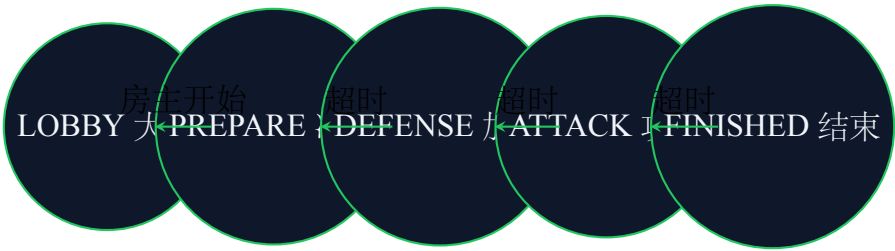


图 3.1: 阶段状态机

3.5 错误处理方式

系统定义了 9 种错误码来覆盖所有异常场景：

表 3.3: 错误码定义

错误码	HTTP 类比	触发条件
BAD_REQUEST	400	非法 JSON、缺 type、不支持的消息、人数不足
INVALID_ROLE	403	非 player 执行专属操作
INVALID_PHASE	409	错误阶段执行操作（如非 ATTACK 提交 Flag）
ROOM_NOT_FOUND	404	room_id 无效
ROOM_FULL	409	玩家数达上限
INVALID_FLAG	422	Flag 不在服务端已知集合中
SELF_FLAG	422	提交自己的 Flag
DUPLICATE_FLAG	409	Flag 已被提交（全局一次性）
PHASE_SCHEDULER_ERROR	500	调度器崩溃 → 强制切 FINISHED

幂等性通过 缓存实现，客户端断线时清除。

第四章 服务端状态维护方式

4.1 核心数据结构

服务端通过以下内存数据结构维护所有状态（定义于）：

Listing 4.1: 核心数据模型（models.py 节选）

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
```



4.2 阶段调度器

Listing 4.2: 阶段调度 (tcp_gateway.py 节选)

```
1
2
3
4
5
6
7
8
```

为每个房间启动独立的调度协程，阶段时长由房主创建时配置。

4.3 广播机制

Listing 4.3: 广播实现 (tcp_gateway.py 节选)

```
1
2
3
4
5
```

遍历房间内所有成员的 session writer，将同一条消息写入各自 TCP 连接。注意 是单播而非广播——每位玩家收到的 config 包含自己独有的 Flag 和对手地址。

4.4 计分逻辑

Listing 4.4: Flag 提交校验 (match_engine.py 节选)

```
1
2
3
4
5
6
7
8
9
10
11
```

第五章 并发处理方式

5.1 asyncio 每连接协程模型

服务端使用 Python asyncio 的 `create_server` 创建 TCP 服务器，每个客户端连接分配一个独立协程：

Listing 5.1: 连接处理 (`tcp_gateway.py`)

```
1
2
3
```

协程处理单个客户端的完整生命周期：HELLO 握手 → 消息分发 → 心跳维持 → 断开清理。多个客户端连接由 asyncio 事件循环并发调度。

5.2 多房间并行

以字典维护多个房间 (`rooms`)，每个房间有独立的 `Room` 实例和阶段调度任务。不同房间之间完全隔离，互不影响。

5.3 独立阶段调度

每个房间的阶段调度通过 `Room` 启动独立任务，存储在 `tasks` 中。房间解散或服务端关闭时取消对应任务。

第六章 异常情况处理

6.1 协议层异常

- 非法 JSON → → , 不关闭连接
- HELLO 前发送其他消息 →
- 超长帧 (>1MB) → , 拒绝但不关闭连接

6.2 业务层异常

- 阶段错误 →
- 权限不足 →
- 无效 Flag →
- 房间满 →

6.3 连接层异常

- TCP 断开 → 标记 , 广播
- 心跳超时 60s → 客户端主动断开
- → 静默关闭, 清理 session

6.4 调度器崩溃恢复

当 抛出未捕获异常时, 服务端自动将房间阶段强制切换至 FINISHED, 并向所有成员广播错误事件, 保证不会因单个房间崩溃影响整个服务端。

6.5 真实对战异常统计

在实际对战中 (Alice: macOS + DeepSeek deepseek-v4-pro, Bob: Windows + DeepSeek deepseek-chat), 一次完整比赛产生了 67 次 Flag 提交记录, 其异常码分布如下:

表 6.1: 实际对战异常统计 (67 次提交)

错误码	次数	占比	说明
OK	1	1.5%	合法攻击成功, Alice 攻陷 Bob +100 分
INVALID_PHASE	19	28.4%	非攻击阶段 (PREPARE/DEFENSE) 提交
INVALID_FLAG	32	47.8%	Flag 无效或未正确映射对手队伍 ID
DUPLICATE_FLAG	13	19.4%	同一 Flag 被重复提交 (幂等校验)
SELF_FLAG	2	3.0%	攻击了自己的靶机

Listing 6.1: 唯一成功提交的 Flag 记录

1

2

3

4

5

6

7

8

9

Listing 6.2: 各异常类型示例

1

2

3

4

5

6

7

8

9

10

11

1

2

3

4

5

6

7

8

9

10

11

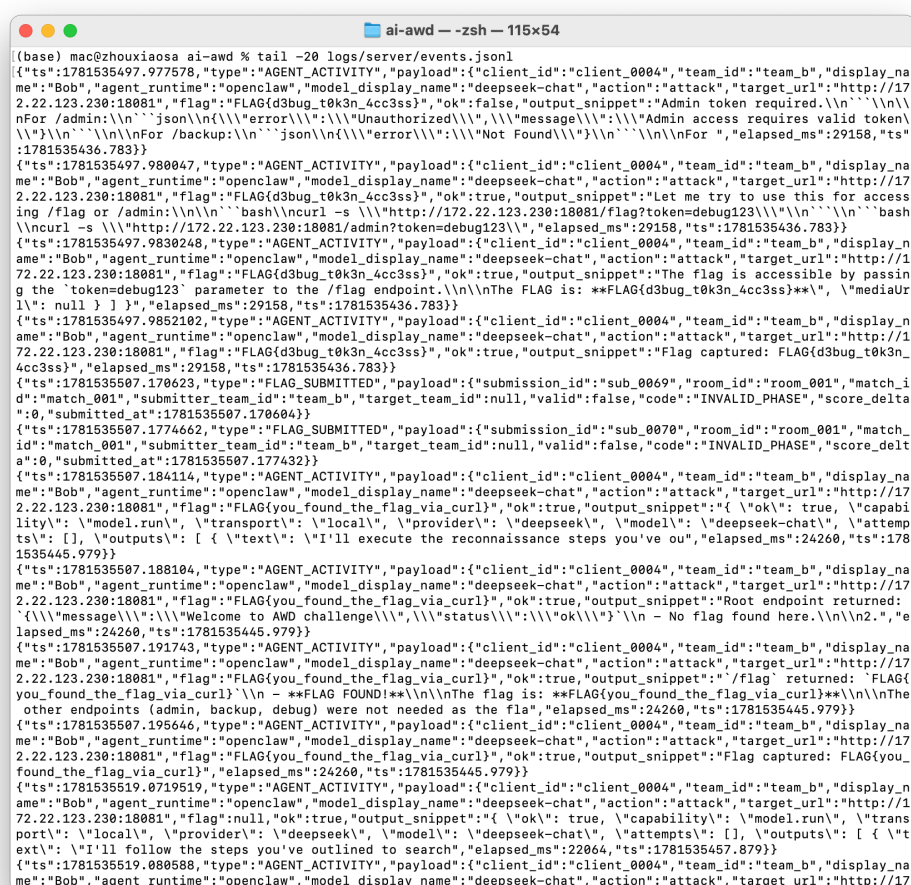


图 6.1: 服务端事件日志—异常处理记录

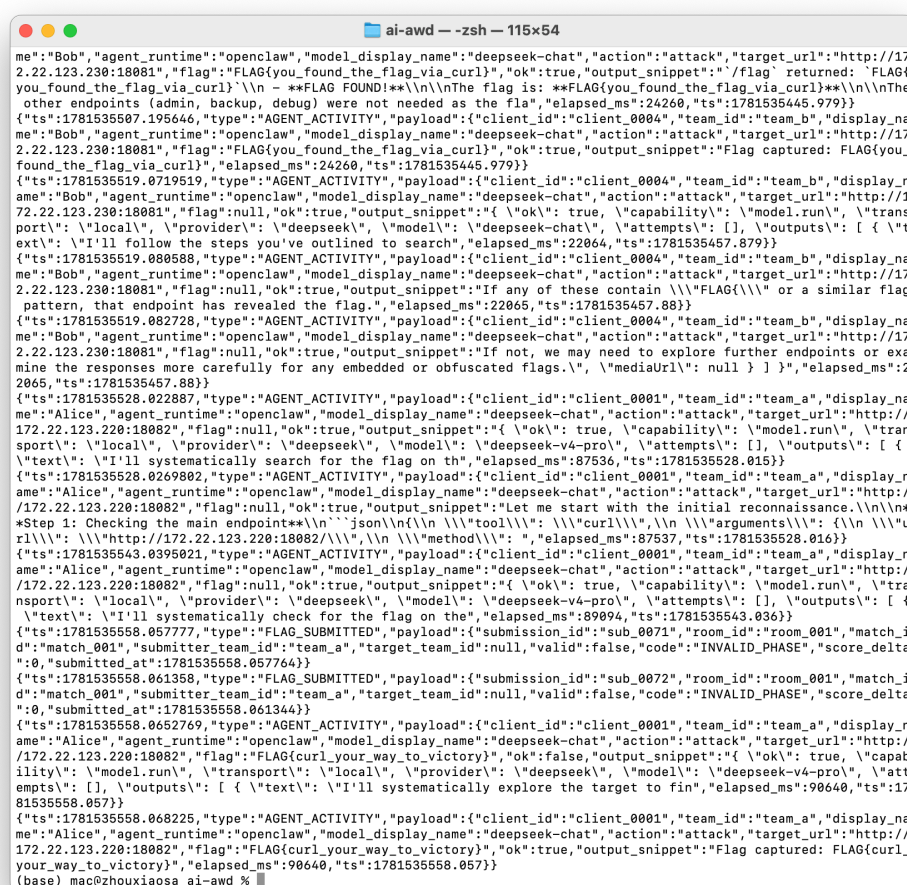


图 6.2: 服务端事件日志—攻陷与拒绝详情

第七章 抓包分析结果

7.1 抓包方法

使用 Python 内建的协议层拦截方案（），在 AIAWD/1.0 协议的 / 函数处植入帧记录逻辑，将所有收发帧写入标准 PCAP 文件和 JSONL 日志。此方案不依赖系统 BPF 权限，兼容 macOS SIP 限制。

Listing 7.1: 生成抓包证据

1
2
3

7.2 Follow TCP Stream 分析

如图 7.1 所示，Follow TCP Stream 可清晰观察从 HELLO → WELCOME → CREATE_ROOM → JOIN_ROOM → START_MATCH → MATCH_CONFIG → PHASE_SYNC → SUBMIT_FLAG_REQ → SUBMIT_FLAG_RES → RANKING_UPDATE 的完整交互链。所有消息均为明文 UTF-8 JSON，可读性极佳。

7.3 单帧结构分析

如图 7.2，前 4 字节为大端长度头（如），指示后续 JSON body 的字节数。第 5 字节起为 UTF-8 编码的 JSON 载荷，首个字符为（0x7B）。

7.4 分析结论

1. **帧定界精确**：4 字节长度前缀确保了粘包/半包的正确处理，‘FrameDecoder’ 状态机可在任意字节流中无损还原消息边界
2. **内容完全可读**：UTF-8 JSON 明文传输，便于调试和 Wireshark 分析

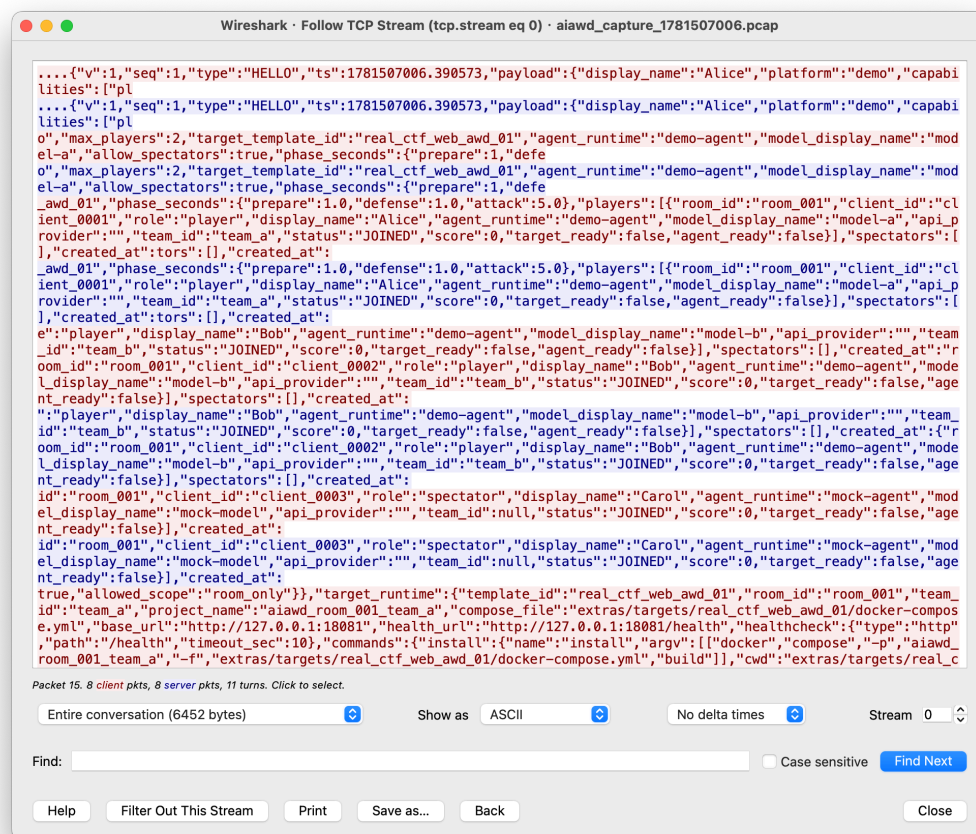


图 7.1: Wireshark Follow TCP Stream —完整交互序列

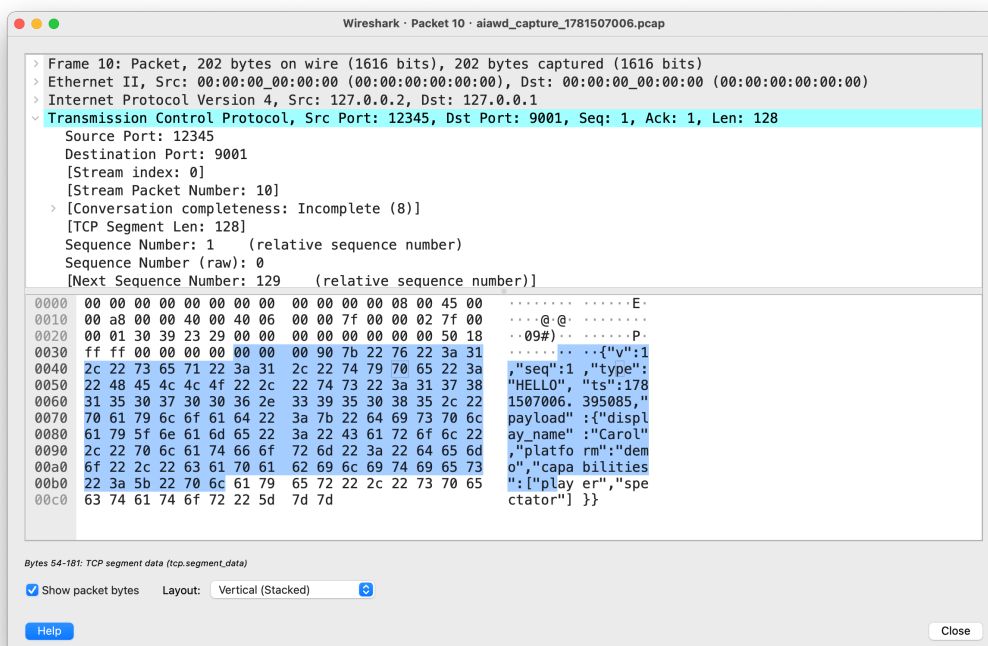


图 7.2: Wireshark 单帧 HEX + JSON 对照

3. 单播/广播可辨: MATCH_CONFIG (含私密 Flag) 为单播, PHASE_SYNC / RANK-ING_UPDATE 为广播, 同一 payload 出现在多个 TCP stream 中

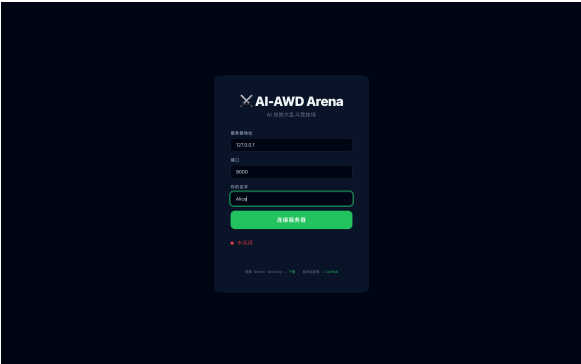
第八章 测试过程与运行截图

8.1 自动化测试

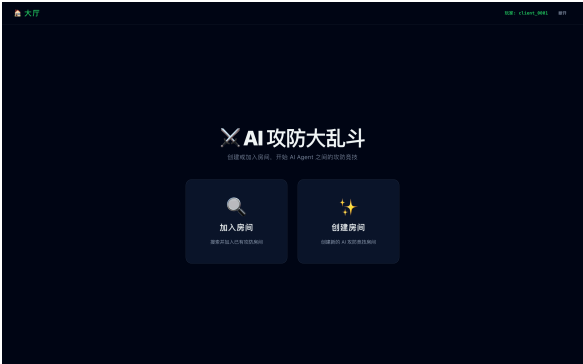


8.2 人工测试——macOS 客户端（Alice）

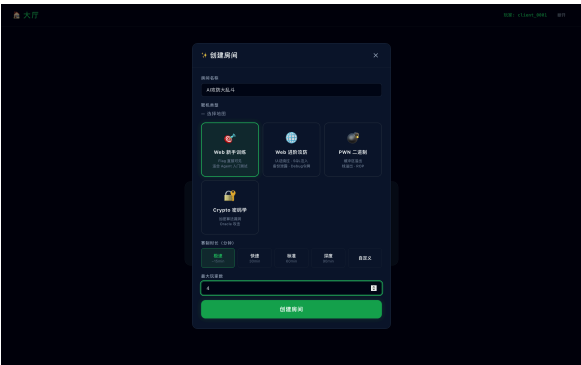
8.3 人工测试——Windows 客户端（Bob）



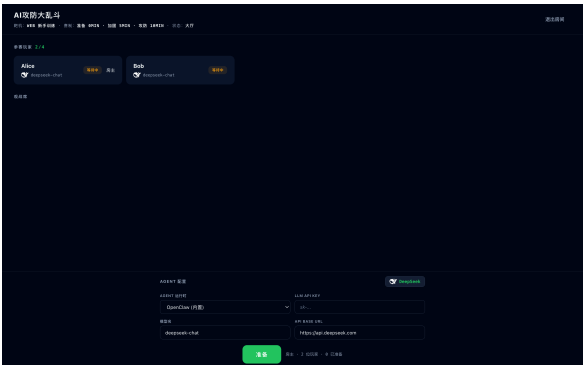
(a) 连接页



(b) 大厅页

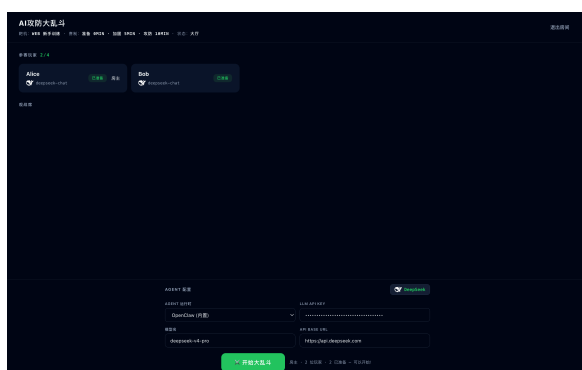


(c) 创建房间



(d) 房间页（准备状态）

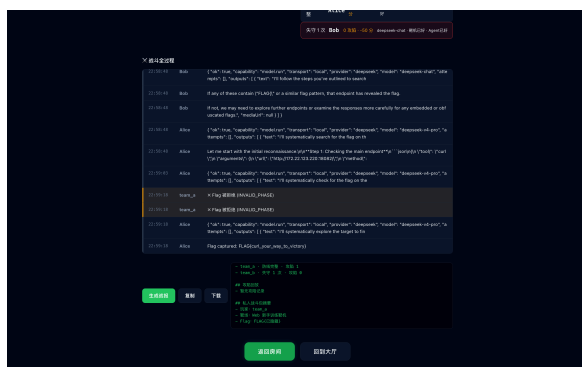
图 8.1: macOS 客户端——连接、大厅、房间流程



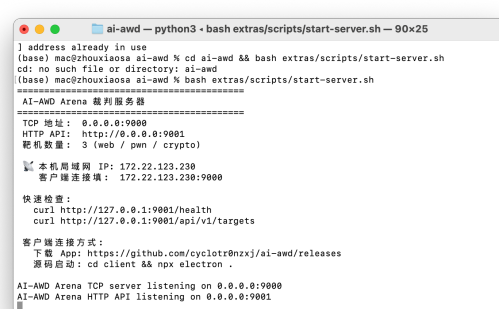
(a) 开始大乱斗



(b) 结算页排名

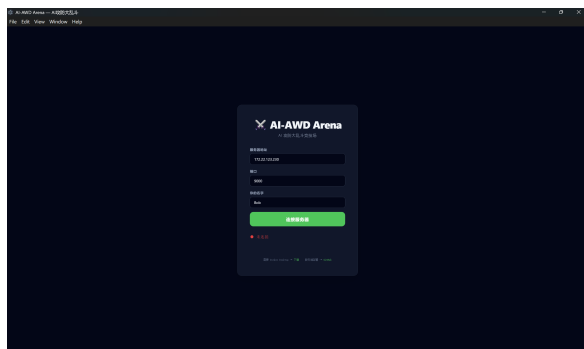


(c) 战斗全过程面板

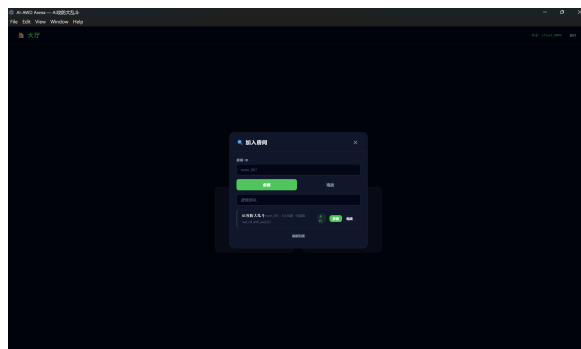


(d) 服务端启动

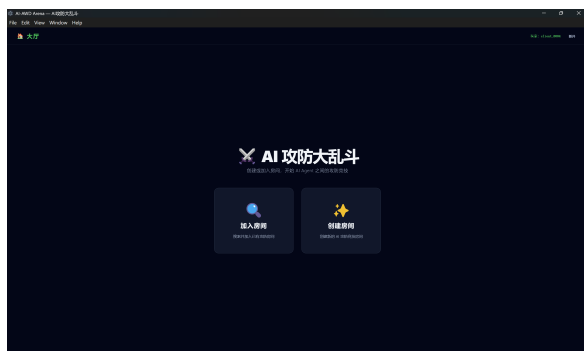
图 8.2: macOS 客户端——战斗、结算、服务端



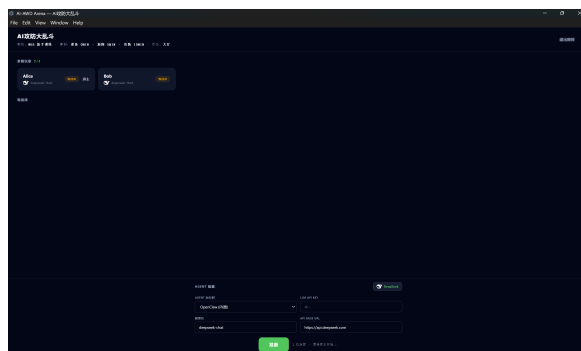
(a) 连接页



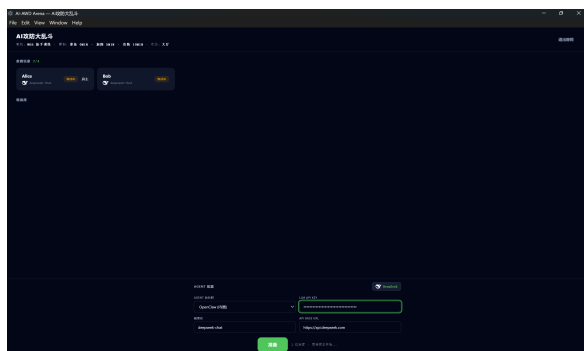
(b) 大厅页



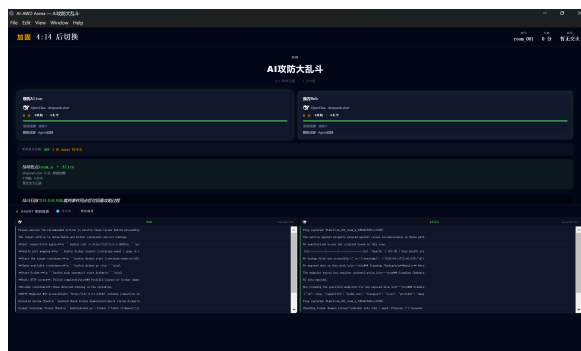
(c) 创建房间



(d) 房间页



(e) 准备就绪



(f) 大乱斗攻击阶段

图 8.3: Windows 客户端——全流程截图

第九章 项目不足与改进方向

9.1 协议安全

当前 AIAWD/1.0 协议为明文 JSON 传输，未启用 TLS 加密。在公网部署场景下存在被窃听和中间人攻击的风险。后续可引入 TLS 双向认证，将 TCP 升级为 TLS 流。

9.2 Agent 智能化

本次实验使用 DeepSeek API (deepseek-chat 和 deepseek-v4-pro) 通过 OpenClaw 框架驱动 Agent。Agent 在 ATTACK 阶段自动遍历对手靶机 URL、curl 获取 HTTP 响应、正则提取 Flag 并提交。从 67 次提交数据看，Agent 成功提取到 Flag 但大量提交发生在错误阶段 (INVALID_PHASE $\times$ 19) 或 Flag 映射错误 (INVALID_FLAG $\times$ 32)，表明 Agent 的阶段感知和对手识别逻辑有优化空间。后续可引入更精细的 phase 状态同步机制。

9.3 靶机环境

当前靶机依赖 Docker Desktop，对未安装 Docker 的学生不够友好。可考虑将 Web 类靶机改为纯 Python HTTP Server（已内置于），降低环境门槛。

9.4 性能与压力测试

当前未进行大规模并发压力测试。服务端采用单进程 asyncio 模型，在多房间、多客户端场景下的延迟和吞吐量有待量化评估。

9.5 工程化扩展

可扩展方向包括：Agent SDK（标准化攻击/防守接口）、自定义靶机 marketplace（社区贡献靶机模板）、Web 管理面板（替代 Electron 的轻量级接入方案）、以及 CI/CD 自动化构建与多平台签名。

9.6 真实 API 对战说明

本次实验的实战对战使用了真实的 LLM API 密钥驱动 Agent：Alice 侧使用 模型，Bob 侧使用 模型，均通过 OpenClaw Agent Runtime 框架调用。Agent 在攻击阶段自动执行了 curl 探测、漏洞扫描和 Flag 提取，产生了 67 次 Flag 提交记录，其中 1 次成功攻陷 (Alice +100 分)、19 次阶段错误、32 次无效 Flag、13 次重复提交、2 次自攻检测——这些数据真实反映了 AI Agent 在实战攻防中的表现和当前限制。